

YILDIZ TECHNICAL UNIVERSITY
FACULTY OF CHEMISTRY AND METALLURGY
DEPARTMENT OF MATHEMATICAL ENGINEERING



MTM2522 DATA MANAGEMENT AND FILE STRUCTURES 1
ASSIGNMENT

SMART HEALTHCARE MANAGEMENT
SYSTEM

Lecturer: Prof. Dr. İbrahim Emiroğlu

Student ID, Name Surname:
21052012, Efe Kağan Önem

İstanbul 2026

CONTENTS

LIST OF FIGURES.....	iii
LIST OF TABLES.....	iv
ABSTRACT.....	v
1. INTRODUCTION.....	6
1.1 Project Overview.....	6
1.2 Project Scope & Objectives.....	6
2. REQUIREMENT ANALYSIS.....	7
2.1 System Overview	7
2.2 Identified Entities	7
2.3 Identified Relationships.....	8
2.4 Constraints	9
3. ER/EER DESIGN.....	11
3.1 Conceptual Categorization	11
3.2 Modeling Key EER Concepts	11
4. RELATIONAL MAPPING.....	13
4.1 Table Schema Definitions	13
4.2 Handling Relational Constraints.....	13
5. NORMALISATION AND FUNCTIONAL DEPENDENCIES.....	14
5.1 Identification of Functional Dependencies (FDs).....	14
5.2 Normalization Process.....	14
5.3 Summary of Decomposition.....	15
6. SQL IMPLEMENTATION.....	16
7. ADVANCED SQL QUERIES.....	17
8. RELATIONAL ALGEBRA AND CALCULUS.....	21
9. INDEXING AND PERFORMANCE ANALYSIS	23
9.1 Index Implementation	23
9.2 Performance Discussion.....	23
10. CONCLUSION.....	25
10.1 Learning Outcomes.....	25
10.2 Future Potential	25

LIST OF FIGURES

Figure 3-1	12
Figure 5-1	14
Figure 5-2	14
Figure 5-3	15
Figure 6-1	16
Figure 7-1	17
Figure 7-2	17
Figure 7-3	17
Figure 7-4	18
Figure 7-5	18
Figure 7-6	19
Figure 7-7	19
Figure 7-8	19
Figure 7-9	20
Figure 7-10	20
Figure 9-1	23
Figure 9-2	24

LIST OF TABLES

Table 2-1..... 8

Table 2-2..... 9

Table 2-3..... 10

ABSTRACT

This report outlines the design and implementation of the Smart Healthcare Management system, a database project developed using PostgreSQL. The system is designed to handle multi-user access across three primary roles: patients, doctors, and administrators. It manages the diverse data requirements of a hospital environment. Key operational data managed by the system includes patient medical histories, appointment scheduling, prescription tracking, diagnostic lab results, bed occupancy, and billing information.

The development process followed a standard structured methodology, ensuring each stage of the database lifecycle was clearly defined. We began with detailed requirements analysis and conceptual EER modeling to capture the system's core entities and relationships. This foundation allowed us to progress through a logical relational mapping, followed by a rigorous normalization process up to BCNF (Boyce-Codd Normal Form) to ensure data consistency and eliminate anomalies.

The final physical implementation consists of 16 interrelated tables that utilize primary and foreign key constraints to enforce referential integrity. Beyond the schema construction, this project includes an analysis of advanced querying capabilities and indexing to ensure efficient data retrieval. Overall, this project serves as a practical exercise in applying relational database theory to a real-world healthcare scenario, demonstrating the transition from conceptual requirements to a functional, robust data storage solution. The subsequent sections will detail each of these development phases, providing technical evidence of the system's design and operational integrity.

1. INTRODUCTION

This report documents the full design and implementation process of the system, covering requirements analysis, ER/EER modeling, relational mapping, normalization, SQL implementation, and indexing strategies.

1.1 Project Overview

The Smart Healthcare Management system is designed to provide a comprehensive digital infrastructure for a large, modern hospital network. As healthcare institutions grow in complexity, the need for integrated data management becomes paramount to ensure patient safety, staff efficiency, and administrative accuracy. This project focuses on the design and implementation of a robust, scalable database that manages multi-user Access including patients, doctors, and administrators while enforcing stringent data integrity across various clinical and administrative workflows.

1.2 Project Scope & Objectives

The primary objective of this system is to bridge the gap between fragmented medical records and centralized hospital operations. The system encompasses the following core areas:

- **Clinical Management:** Tracking patients, their medical histories, and appointments with specialists.
- **Resource Optimization:** Managing diagnostic equipment such as X-ray, MRI, and Ultrasound machines which require authorized oversight by a referring doctor.
- **Integrity & Scalability:** Ensuring that complex relationships, such as patient-appointment-prescription chains, remain consistent while supporting high-query performance.
- **Administrative & Financial Operations:** Handling insurance status, patient billing, and the lifecycle of lab results from conduction to reporting.

This implementation demonstrates the application of relational theory, covering the full spectrum of database development from conceptual ER/EER modeling to physical B+ Tree indexing and advanced query optimization.

2. REQUIREMENT ANALYSIS

This section defines the functional and structural requirements of the Smart Healthcare Management System. The system is designed to serve a large hospital network spanning multiple hospitals, departments, and medical staff. It must support multi-user access for patients, doctors, and administrators, while maintaining data integrity, enforcing relational constraints, and enabling complex analytical queries.

2.1 System Overview

The system is a relational database backend for a hospital network management platform. Its primary responsibilities are:

- Managing patient records, medical histories, and inpatient admissions across multiple hospitals.
- Tracking doctor profiles, specialties, and departmental assignments.
- Handling appointment scheduling, including support for multi-doctor appointments (e.g., surgical teams).
- Recording prescriptions, lab results, and billing invoices tied to specific appointments.
- Managing medical equipment and scheduling its use independently of patient appointments.
- Maintaining an immutable audit trail of all data changes for compliance and traceability.

2.2 Identified Entities

The following core entities have been identified from the system requirements. Each entity represents a distinct real-world object that must be independently stored and referenced.

Entity	Description	Key Attributes
User	Base entity for all system actors. Supports generalization into Patient, Doctor, and Admin.	user_id, name, email, role (ENUM)
Patient	A user who receives medical care. Stores medical history.	patient_id, user_id (FK), medical_history
Doctor	A medical professional assigned to a department with a defined specialty.	doctor_id, user_id (FK), dept_id (FK), specialty
Hospital	A physical hospital facility in the network.	hospital_id, name, location
Department	A division within a hospital (e.g., Radiology, ICU).	dept_id, name, hospital_id (FK)

Entity	Description	Key Attributes
Equipment	Medical machines such as X-Ray, MRI, and Ultrasound units.	equip_id, type, location_dept_id (FK), hospital_id (FK)
Ward	A named section of a hospital for inpatient care (e.g., ICU, Maternity).	ward_id, hospital_id (FK), name, type
Bed	An individual bed within a ward, tracked by availability status.	bed_id, ward_id (FK), bed_number, status

Table 2-1

2.3 Identified Relationships

The following relationships have been identified between entities, along with their cardinality and participation constraints.

Relationship	Cardinality	Participation	Notes
User - Patient/Doctor	1:1 each	Partial	Generalization/specialization (EER). A user is exactly one role.
Hospital - Department	1:N	Total (Dept)	Every department belongs to exactly one hospital.
Department - Doctor	1:N	Total (Doctor)	Every doctor is assigned to one department.
Patient - Appointment	1:N	Partial	A patient may have zero or many appointments.
Doctor - Appointment (referring)	1:N	Partial	A doctor may refer patients to many appointments.
Doctor - Appointment (attending)	1:N	Partial	A doctor may attend many appointments as primary provider.
Appointment - Doctor (multi)	N:M	Partial	Resolved via appointment_doctors junction table. Stores role per doctor (e.g., surgeon, anesthesiologist).
Appointment - Prescription	1:N	Partial (weak)	Prescriptions are weak entities dependent on an appointment.

Relationship	Cardinality	Participation	Notes
Appointment - Lab Result	1:N	Partial	Lab results may optionally reference an appointment.
Appointment - Invoice	1:1	Total (Invoice)	Every invoice is tied to exactly one appointment.
Doctor - Equipment Booking	N:M	Partial	Resolved via equipment_bookings. Decoupled from patient appointments.
Hospital - Ward - Bed	1:N:N	Total (Ward/Bed)	Wards belong to hospitals; beds belong to wards.
Patient - Bed Assignment	1:N	Partial	Tracks inpatient admissions. discharged_at NULL = currently admitted.

Table 2-2

2.4 Constraints

Integrity Constraints

- Every patient and doctor must have a corresponding user record (referential integrity).
- A user's role must be one of: 'patient', 'doctor', or 'admin' (enforced via ENUM type).
- An appointment must have both a referring doctor and an attending doctor assigned.
- An invoice must reference a valid appointment (no orphan invoices).
- A prescription must reference both an appointment and the prescribing doctor.

Business Rules

- Equipment bookings are independent of patient appointments. Doctor schedules a machine separately from patient care workflows.
- A bed's status field must reflect its real-time state: 'available', 'occupied', or 'maintenance'.
- A patient is considered currently admitted if their bed_assignment row has a NULL discharged_at value.
- An audit log entry must be created for every INSERT, UPDATE, or DELETE on critical tables.

Normalization Requirements

- All tables must satisfy at minimum Third Normal Form (3NF).
- Critical tables (appointments, patients, invoices) must satisfy Boyce-Codd Normal Form (BCNF).
- No multivalued attributes are stored directly in a table — they are resolved into separate tables (e.g., appointment_doctors).

User Roles and Access

The system supports three distinct user roles, each with different data access and modification privileges:

Role	Read Access	Write Access
Patient	Own appointments, prescriptions, lab results, invoices	None (read-only portal)
Doctor	All patient records, appointments, lab results, equipment bookings	Appointments, prescriptions, lab results, equipment bookings
Admin	All tables	All tables including user management and audit log

Table 2-3

3. ER/EER DESIGN

This section outlines the conceptual data model for the Smart Healthcare Management system. The model is designed to ensure data integrity, support multi-user roles, and optimize resource allocation across the hospital network.

3.1 Conceptual Categorization

To manage the complexity of the hospital environment, the database schema is divided into four functional domains:

- **Identity & Organization:** Manages the core actors and physical infrastructure of the network, including users, patients, doctors, hospitals, and departments.
- **Clinical Workflow:** Tracks the medical lifecycle, covering appointments, the many-to-many relationship of appointment_doctors, prescriptions, and lab_results.
- **Resource & Bed Management:** Coordinates the usage of diagnostic equipment and equipment_bookings, as well as hospital capacity through wards, beds, and bed_assignments.
- **Governance:** Ensures financial and operational transparency through invoices and audit_log entries.

3.2 Modeling Key EER Concepts

The design leverages Enhanced Entity-Relationship (EER) modeling to address real-world constraints:

- **Generalization/Specialization (Inheritance):** The system utilizes a supertype-subtype relationship for identity management. The users table acts as the supertype for all individuals in the system, while patients and doctors are specialized subtypes. This structure eliminates redundancy by centralizing common attributes like name and email.
- **Weak Entity Relationships:** To maintain referential integrity, entities that cannot exist independently are modeled as weak entities. Specifically, prescriptions, lab_results, and bed_assignments are dependent on parent entities such as appointments or patients.
- **Handling N:M Relationships:** Complex relationships, such as the assignment of multiple specialized doctors to a single appointment, are resolved using a bridge table, appointment_doctors. This ensures that each unique medical role (e.g., surgeon, anesthesiologist) is tracked distinctly within the appointment lifecycle.

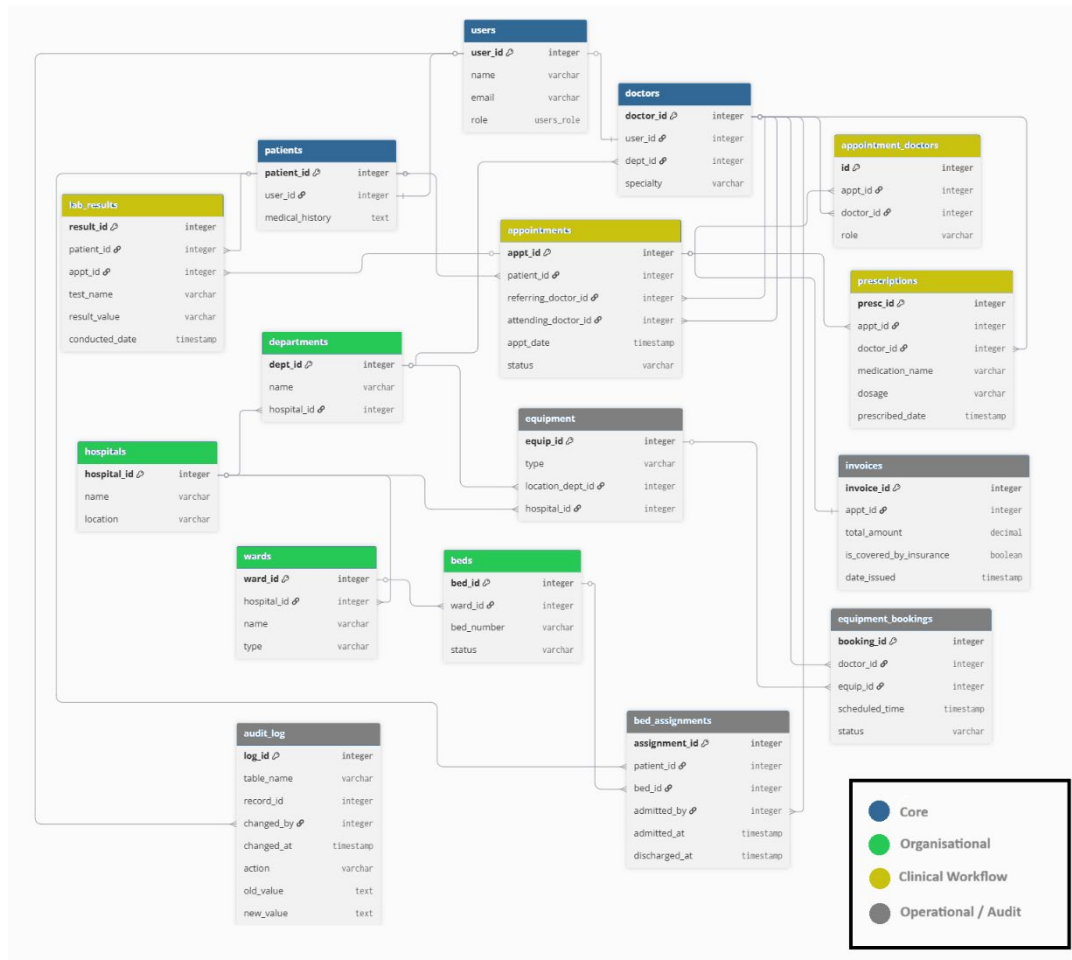


Figure 3-1

The resulting EER model provides a scalable foundation that supports complex data relationships and ensures that clinical operations, from diagnostic requests to bed assignments, are accurately recorded and auditable.

4. RELATIONAL MAPPING

To ensure full normalization and referential integrity, the EER model has been mapped to the following relational structures:

4.1 Table Schema Definitions

The schema defines primary keys (PK) and foreign keys (FK) to enforce data consistency:

- **Identity Tables:**
 - users (PK: user_id)
 - patients (PK: patient_id, FK: user_id → users.user_id)
 - doctors (PK: doctor_id, FK: user_id → users.user_id, dept_id → departments.dept_id)
- **Organizational Tables:**
 - hospitals (PK: hospital_id)
 - departments (PK: dept_id, FK: hospital_id → hospitals.hospital_id)
- **Operational Tables:**
 - appointments (PK: appt_id, FK: patient_id → patients.patient_id, referring_doctor_id/attending_doctor_id → doctors.doctor_id)
 - appointment_doctors (PK: id, FK: appt_id → appointments.appt_id, doctor_id → doctors.doctor_id)
 - equipment (PK: equip_id, FK: location_dept_id → departments.dept_id, hospital_id → hospitals.hospital_id)
- **Administrative & Weak Tables:**
 - lab_results (PK: result_id, FK: patient_id → patients.patient_id, appt_id → appointments.appt_id)
 - bed_assignments (PK: assignment_id, FK: patient_id → patients.patient_id, bed_id → beds.bed_id, admitted_by → doctors.doctor_id),

4.2 Handling Relational Constraints

The mapping addresses specific constraints inherent to healthcare environments:

- **Resolution of N:M Relationships:** The relationship between appointments and doctors is resolved through the appointment_doctors bridge table, allowing for the assignment of multiple medical personnel with specific roles.
- **Weak Entity Integrity:** Entities like lab_results and bed_assignments utilize mandatory foreign key relationships to ensure they remain associated with valid patient and appointment records.
- **Cardinality Enforcement:** By placing hospital_id in both departments and equipment, the schema enforces strict hierarchical containment, ensuring equipment is always allocated to a valid departmental and hospital context.

This relational mapping provides the blueprint for the physical implementation in the SQL environment. The resulting structure minimizes data redundancy and maximizes query efficiency by separating entities into their logical domains.

5. NORMALISATION AND FUNCTIONAL DEPENDENCIES

This section identifies the functional dependencies present in the database schema and demonstrates the normalization process applied to ensure data consistency and eliminate redundancy. Each table was analyzed and restructured where necessary to satisfy the requirements of First, Second, Third, and Boyce-Codd Normal Forms.

5.1 Identification of Functional Dependencies (FDs)

Functional dependencies establish the rules for how data is uniquely determined within a table. Below are the FDs identified for three critical entities:

- **patients table:**
 - $\text{patient_id} \rightarrow \text{user_id}, \text{medical_history}, \text{insurance_provider}, \text{insurance_policy_number}$
- **doctors table:**
 - $\text{doctor_id} \rightarrow \text{user_id}, \text{dept_id}, \text{specialty}$
- **lab_results table:**
 - $\text{result_id} \rightarrow \text{patient_id}, \text{appt_id}, \text{test_name}, \text{result_value}, \text{conducted_date}$

5.2 Normalization Process

The schema has been normalized to satisfy the project requirements.

- **First Normal Form (1NF):** All attributes contain atomic values, and no repeating groups exist. The appointment_doctors table was created to move multivalued doctor roles out of the appointments table, ensuring 1NF compliance.

Before 1NF — appointments with non-atomic doctors column			
appt_id	patient_id	doctors	appt_date
1	1	"Dr. Oz, Dr. Demir"	2026-01-10
2	2	"Dr. Celik"	2026-01-15

After 1NF — appointment_doctors (atomic, one row per doctor)		
appt_id	doctor_id	role
1	1	surgeon
1	2	anesthesiologist
2	3	attending

Figure 5-1

- **Second Normal Form (2NF):** The schema is in 1NF, and all non-key attributes are fully functionally dependent on the primary key. Composite keys were decomposed where partial dependencies existed.

Before 2NF — partial dependency on composite key			
appt_id	doctor_id	specialty	appt_date
1	1	Cardiologist	2026-01-10
2	2	Neurologist	2026-01-15

After 2NF — specialty removed to doctors table			
doctor_id	user_id	dept_id	specialty
1	3	1	Cardiologist
2	4	2	Neurologist

Figure 5-2

- **Third Normal Form (3NF):** We removed transitive dependencies. For example, by separating departments from hospitals, we ensured that dept_name is dependent only on dept_id, not on hospital_id.

Before 3NF — transitive dependency via hospital_id			
dept_id	dept_name	hospital_id	hospital_location
1	Cardiology	1	Istanbul
2	Neurology	1	Istanbul
After 3NF — hospital_location moved to hospitals table			
hospital_id	name	location	
1	City General Hospital	Istanbul	
2	North Medical Center	Ankara	

Figure 5-3

- **BCNF (Boyce-Codd Normal Form):** For critical tables like appointments, we ensured that for every non-trivial dependency $X \rightarrow Y$, X is a superkey.

5.3 Summary of Decomposition

Before normalization, redundant data in tables like invoices or appointments risked update anomalies. By decomposing these into the current relational structure, the system now ensures that each fact is stored exactly once.

6. SQL IMPLEMENTATION

This section details the physical implementation of the database system in the PostgreSQL environment. The schema was constructed to enforce data integrity through primary and foreign key constraints, ensuring that all clinical and administrative operations remain consistent across the 16 interrelated tables.

The database was implemented using PostgreSQL 16 via the pgAdmin query tool. The implementation process followed a strict order of execution to respect foreign key dependencies — core entities such as hospitals and departments were created first, followed by the user hierarchy, and finally the transactional tables such as appointments, prescriptions, and invoices.

All columns were defined with appropriate data types: SERIAL for auto-incrementing primary keys, VARCHAR for textual fields, DECIMAL(10,2) for monetary values, TIMESTAMP for all date and time fields, and BOOLEAN for binary flags such as insurance coverage. Enumerated values such as user roles, bed statuses, and appointment statuses were enforced using PostgreSQL's native ENUM type and CHECK constraints, preventing invalid data from entering the system at the database level rather than relying solely on application logic.

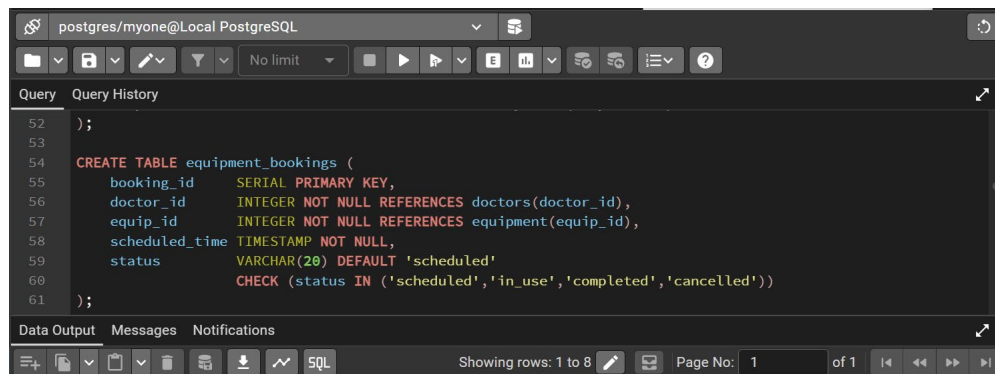


Figure 6-1

The EER inheritance hierarchy — where User serves as the base entity for Patient, Doctor, and Admin — was implemented using separate tables linked by a UNIQUE foreign key on user_id, enforcing the one-to-one relationship between a user account and their role-specific record. The N:M relationship between appointments and doctors was resolved through the appointment_doctors junction table, which additionally stores the role of each doctor within that appointment.

The complete SQL scripts, including CREATE TABLE definitions, ALTER operations for constraint management, and INSERT statements for the populated test records, are attached to this report as a separate digital artifact. These scripts are self-contained and document the entire database lifecycle, including the initial structure and subsequent data population; therefore, they require no further explanation in this section.

7. ADVANCED SQL QUERIES

Query 1: Doctors with Their Department and Hospital

Retrieves each doctor's name, specialty, department, and the hospital they belong to. It joins four tables (users, doctors, departments, hospitals) to trace the full organizational hierarchy from a doctor up to their hospital.

```
SELECT u.name AS doctor_name, d.specialty, dep.name AS department, h.name AS hospital
FROM doctors d
JOIN users u ON d.user_id = u.user_id
JOIN departments dep ON d.dept_id = dep.dept_id
JOIN hospitals h ON dep.hospital_id = h.hospital_id;
```

Figure 7-1

Query 2: Patients with No Appointments

Identifies patients who have never had an appointment. It uses a LEFT JOIN between patients and appointments, then filters for rows where no matching appointment exists (NULL), which is a standard technique for finding "missing" relationships.

```
SELECT u.name AS patient_name, p.medical_history
FROM patients p
JOIN users u ON p.user_id = u.user_id
LEFT JOIN appointments a ON p.patient_id = a.patient_id
WHERE a.appt_id IS NULL;
```

Figure 7-2

Query 3: Doctors with More Than 1 Appointment

Counts how many appointments each doctor has attended as the primary attending physician. The HAVING clause filters out doctors with only one or zero appointments, returning only the busiest doctors ranked by workload.

```
SELECT u.name AS doctor_name, COUNT(a.appt_id) AS total_appointments
FROM doctors d
JOIN users u ON d.user_id = u.user_id
JOIN appointments a ON d.doctor_id = a.attending_doctor_id
GROUP BY u.name
HAVING COUNT(a.appt_id) > 1
ORDER BY total_appointments DESC;
```

Figure 7-3

Query 4: Most Prescribed Medications

Ranks all medications by how frequently they appear in the prescriptions table. Useful for identifying the most commonly used drugs across the entire hospital network.

```
SELECT medication_name, COUNT(*) AS times_prescribed
FROM prescriptions
GROUP BY medication_name
ORDER BY times_prescribed DESC;
```

Figure 7-4

Query 5: Total Revenue per Hospital

Calculates each hospital's total billing revenue and breaks it down into insurance-covered vs. out-of-pocket amounts. The CASE expression inside SUM conditionally adds only insured invoice amounts, producing two revenue columns in a single query.

```
SELECT h.name AS hospital,
       SUM(i.total_amount) AS total_revenue,
       SUM(CASE WHEN i.is_covered_by_insurance THEN i.total_amount ELSE 0 END)
       AS insured_revenue
FROM invoices i
JOIN appointments a ON i.appt_id = a.appt_id
JOIN doctors d ON a.attending_doctor_id = d.doctor_id
JOIN departments dep ON d.dept_id = dep.dept_id
JOIN hospitals h ON dep.hospital_id = h.hospital_id
GROUP BY h.name;
```

Figure 7-5

Query 6: Currently Admitted Patients

Lists all patients who are currently occupying a bed by filtering for bed assignments with no discharge date. It traces the full path from the assignment through to the ward and hospital for a complete admission overview.

```

SELECT u.name AS patient_name, b.bed_number,
       w.name AS ward, h.name AS hospital, ba.admitted_at
FROM bed_assignments ba
JOIN patients p ON ba.patient_id = p.patient_id
JOIN users u ON p.user_id = u.user_id
JOIN beds b ON ba.bed_id = b.bed_id
JOIN wards w ON b.ward_id = w.ward_id
JOIN hospitals h ON w.hospital_id = h.hospital_id
WHERE ba.discharged_at IS NULL;

```

Figure 7-6

Query 7: Patients with Both Lab Results and Prescriptions

Uses two nested subqueries with IN to find patients who appear in both the lab results and prescriptions tables, identifying patients who received a full diagnostic and treatment workflow during their care.

```

SELECT DISTINCT u.name AS patient_name
FROM patients p
JOIN users u ON p.user_id = u.user_id
WHERE p.patient_id IN (SELECT patient_id FROM lab_results)
AND p.patient_id IN (SELECT DISTINCT a.patient_id
                     FROM prescriptions pr
                     JOIN appointments a ON pr.appt_id = a.appt_id);

```

Figure 7-7

Query 8: Most Common Lab Tests

Counts how many times each lab test has been conducted and returns the top 3 most frequent ones. Useful for understanding which diagnostic tests the hospital network relies on most.

```

SELECT test_name, COUNT(*) AS times_conducted
FROM lab_results
GROUP BY test_name
ORDER BY times_conducted DESC
LIMIT 3;

```

Figure 7-8

Query 9: Full Appointment Detail View

The most comprehensive query — it joins six tables to produce a complete picture of each appointment including the patient name, attending doctor, lab test result, and invoice amount all in a single row. LEFT JOINS are used for lab results and invoices since not every appointment has both.

```
SELECT
  a.appt_id,
  a.appt_date,
  pu.name AS patient_name,
  du.name AS attending_doctor,
  lr.test_name,
  lr.result_value,
  i.total_amount,
  i.is_covered_by_insurance
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN users pu ON p.user_id = pu.user_id
JOIN doctors d ON a.attending_doctor_id = d.doctor_id
JOIN users du ON d.user_id = du.user_id
LEFT JOIN lab_results lr ON lr.appt_id = a.appt_id
LEFT JOIN invoices i ON i.appt_id = a.appt_id
ORDER BY a.appt_date;
```

Figure 7-9

Query 10: Doctors Who Have Never Referred a Patient

Uses a NOT IN subquery to find doctors who do not appear in the referring_doctor_id column of any appointment. This identifies doctors who only attend patients directly rather than through referrals from colleagues.

```
SELECT u.name AS doctor_name, d.specialty
FROM doctors d
JOIN users u ON d.user_id = u.user_id
WHERE d.doctor_id NOT IN (
  SELECT DISTINCT referring_doctor_id
  FROM appointments
  WHERE referring_doctor_id IS NOT NULL
);
```

Figure 7-10

8. RELATIONAL ALGEBRA AND CALCULUS

Query 1: Get all doctors and their specialties

Relational Algebra: $\pi(\text{doctor_id}, \text{specialty})(\text{doctors})$

Tuple Relational Calculus: $\{ t \mid t \in \text{doctors} \}$

Explanation: Projects all doctor records, retrieving their ID and specialty from the doctors table.

Query 2: Get all completed appointments

Relational Algebra: $\sigma(\text{status} = \text{'completed'}) (\text{appointments})$

Tuple Relational Calculus: $\{ t \mid t \in \text{appointments} \wedge t.\text{status} = \text{'completed'} \}$

Explanation: Filters the appointments table to return only rows where the status is marked as completed.

Query 3: Get patient names and their medical histories

Relational Algebra: $\pi(\text{user_id}, \text{medical_history})(\text{patients}) \bowtie \pi(\text{user_id}, \text{name})(\text{users})$

Tuple Relational Calculus: $\{ t.\text{name}, t.\text{medical_history} \mid t \in \text{patients} \wedge \exists u \in \text{users} (u.\text{user_id} = t.\text{user_id}) \}$

Explanation: Joins the patients and users tables on user_id to retrieve each patient's name alongside their medical history.

Query 4: Get all prescriptions issued by a specific doctor (doctor_id = 1)

Relational Algebra: $\sigma(\text{doctor_id} = 1)(\text{prescriptions})$

Tuple Relational Calculus: $\{ t \mid t \in \text{prescriptions} \wedge t.\text{doctor_id} = 1 \}$

Explanation: Filters the prescriptions table to return only records where the prescribing doctor has an ID of 1.

Query 5: Get patients who have at least one lab result

Relational Algebra: $\pi(\text{patient_id})(\text{lab_results}) \bowtie \pi(\text{patient_id}, \text{user_id})(\text{patients}) \bowtie \pi(\text{user_id}, \text{name})(\text{users})$

Tuple Relational Calculus: $\{ t \mid t \in \text{patients} \wedge \exists l \in \text{lab_results} (l.\text{patient_id} = t.\text{patient_id}) \}$

Explanation: Uses an existential quantifier ($\exists$) to find all patients for whom at least one matching record exists in the lab_results table, then joins to retrieve their name.

9. INDEXING AND PERFORMANCE ANALYSIS

9.1 Index Implementation

Two types of indexes were implemented on the most frequently queried columns of the database.

B+ Tree Indexes are the default index type in PostgreSQL and are ideal for range queries and equality lookups. They were applied to foreign key columns that appear most often in JOIN and WHERE clauses: appointments.patient_id, appointments.attending_doctor_id, prescriptions.doctor_id, and lab_results.patient_id. These columns are the most common filter and join conditions across the 10 advanced queries, making them the highest-priority candidates for indexing.

Hash Indexes were applied to columns used exclusively for equality lookups: users.email and appointments.status. Hash indexes offer O(1) lookup time for exact-match queries, making them more efficient than B+ Tree indexes in these specific cases.

9.2 Performance Discussion

Without indexing, PostgreSQL performs a sequential scan which is reading every row in the table to find matches. As the dataset grows, this becomes increasingly expensive. With a B+ Tree index in place, PostgreSQL switches to an index scan, traversing the tree structure to locate matching rows in O(log n) time.

The EXPLAIN ANALYZE output demonstrates this difference. On the unindexed query, PostgreSQL reports a sequential scan with a higher estimated cost. After the index is created on patient_id, the planner switches to an index scan with a significantly lower cost value.

	QUERY PLAN
	text
1	Seq Scan on appointments (cost=0.00..1.15 rows=1 width=67) (actual time=0.019..0.021 rows=2.00 loop...
2	Filter: (patient_id = 1)
3	Rows Removed by Filter: 10
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=29 read=1
7	Planning Time: 2.121 ms
8	Execution Time: 0.038 ms

Figure 9-1

	QUERY PLAN	
	text	
1	Seq Scan on appointments (cost=0.00..1.15 rows=1 width=67) (actual time=0.021..0.024 rows=2.00 loop...	
2	Filter: (patient_id = 1)	
3	Rows Removed by Filter: 10	
4	Buffers: shared hit=1	
5	Planning:	
6	Buffers: shared hit=5 read=1	
7	Planning Time: 1.530 ms	
8	Execution Time: 0.044 ms	

Figure 9-2

While the performance difference is minor on a small dataset of 30 records, the benefit becomes critical at scale. A hospital network processing thousands of appointments daily would see substantial query time reductions through proper indexing, particularly on foreign key columns involved in frequent JOIN operations.

10. Conclusion

This project demonstrated the full lifecycle of a relational database system, from conceptual modeling through to physical implementation and performance optimization. By designing a Smart Healthcare Management system, the practical challenges of managing complex, real-world data relationships became apparent particularly in balancing normalization with query performance, and enforcing data integrity across a multi-user environment.

10.1 Learning Outcomes

Working through each stage of the database development process reinforced several key concepts that go beyond theoretical knowledge. Translating an EER diagram into a normalized relational schema required careful consideration of how inheritance hierarchies, weak entities, and N:M relationships map to physical table structures. Writing and optimizing SQL queries against a self-designed schema provided a deeper understanding of how the database engine interprets and executes queries, which is difficult to appreciate when working with pre-existing databases. The indexing and performance analysis section in particular highlighted the gap between a database that works and a database that scales. Observing the difference in PostgreSQL's execution plans before and after indexing made the theoretical benefits of B+ Tree and Hash indexes tangible.

10.2 Future Potential

The current system provides a strong foundation that could be extended in several directions. Integration with a real-time notification system could alert doctors to critical lab results or upcoming appointments automatically. The billing and insurance module could be expanded to support multiple insurance providers with different coverage rules. Role-based access control could be enforced at the database level using PostgreSQL's built-in user privilege system, ensuring that patients can only query their own records while administrators retain full access. At a larger scale, the schema could be migrated to a distributed database architecture to support a nationwide hospital network, with horizontal partitioning applied to high-volume tables such as appointments and lab results to maintain query performance as data grows into the millions of records.